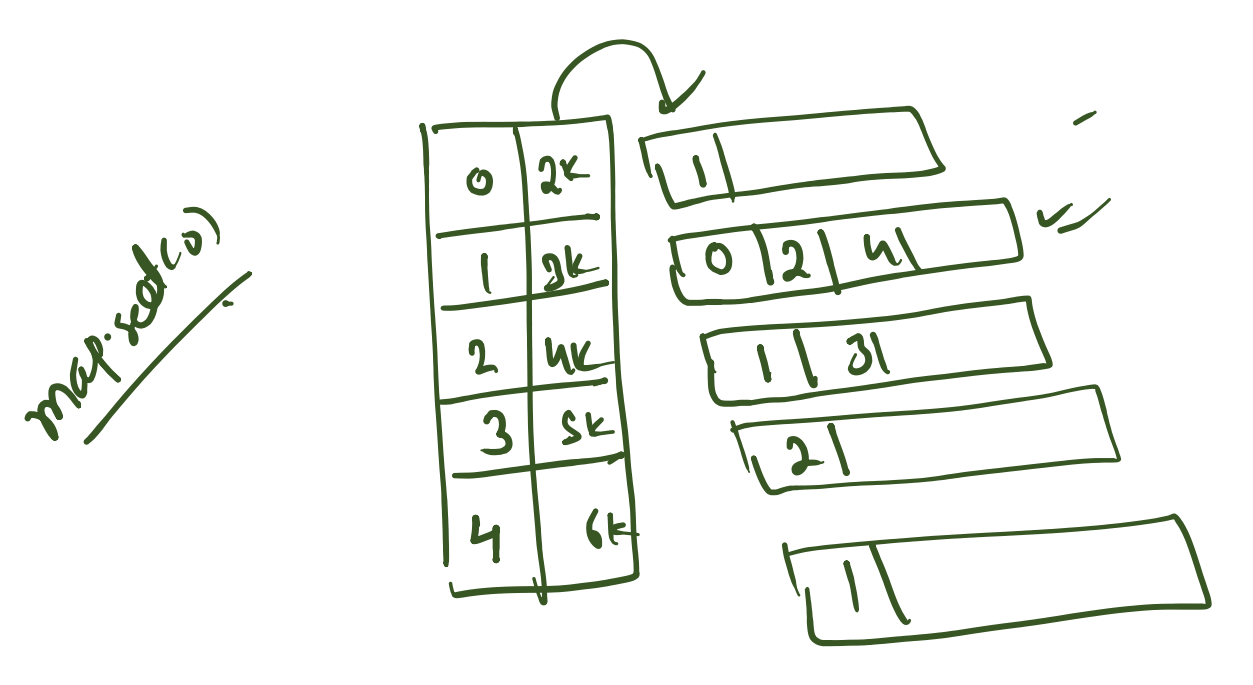
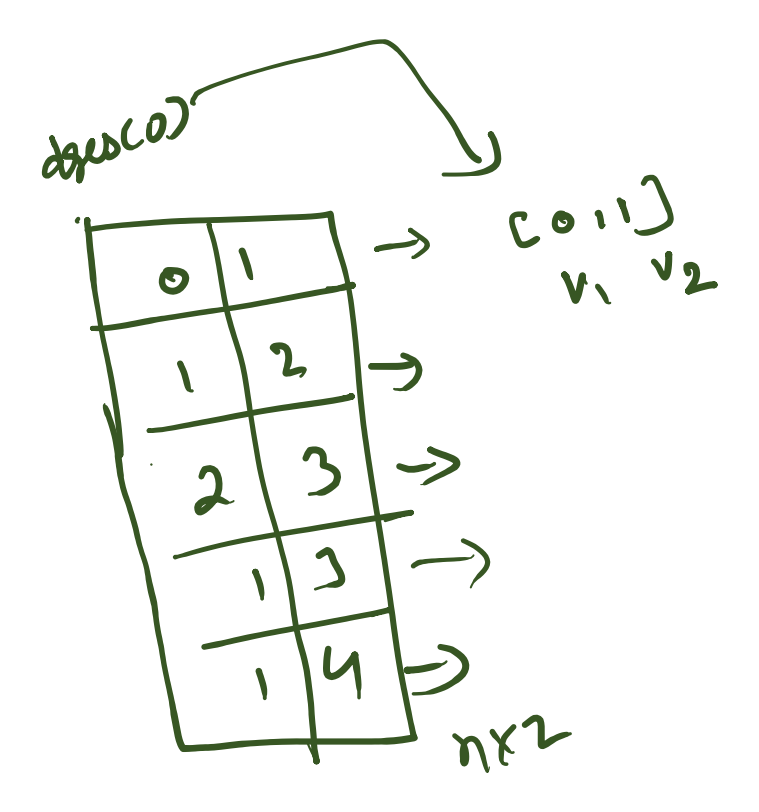
Lec-38

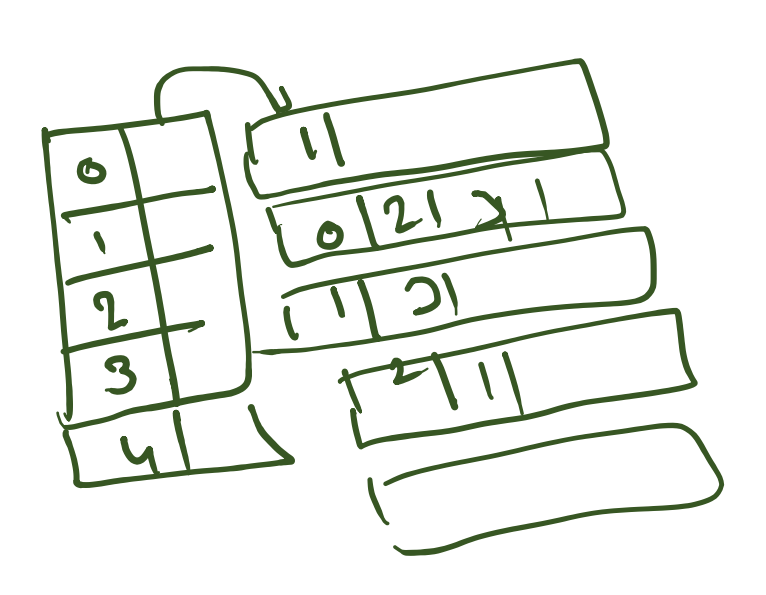
18/07/26 10:15 AM

Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]

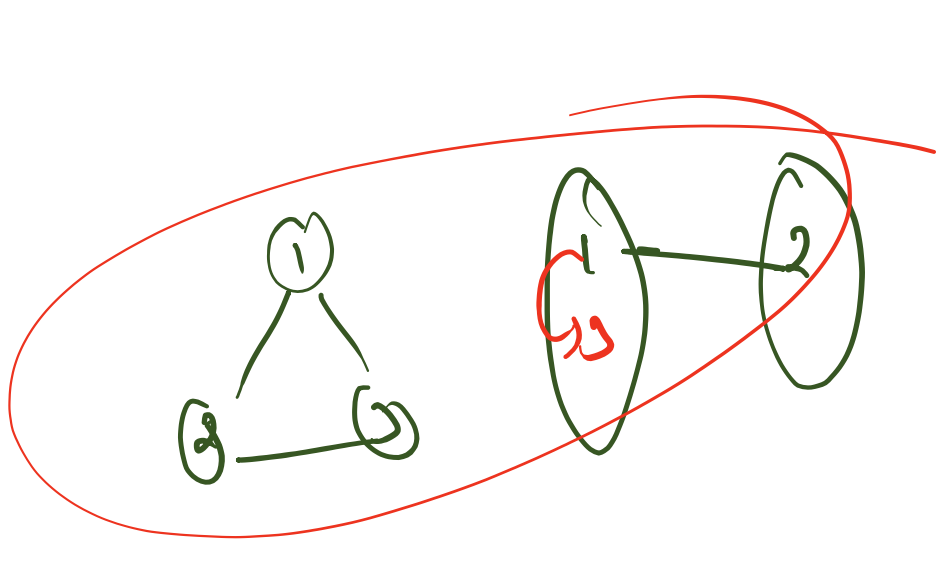

map.put(0)
0 2k
1 3k
2 4k
3 5k
4 6k
1
0 2 4
1 3
2
1

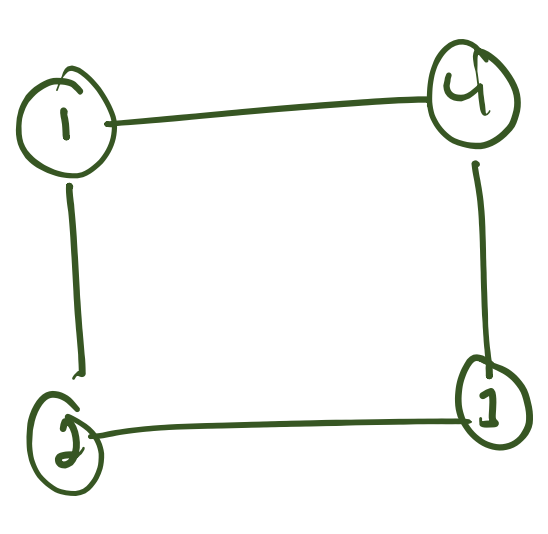

edges[0]
0 1
1 2
2 3
1 3
1 4
[0,1]
v1 v2
n×2

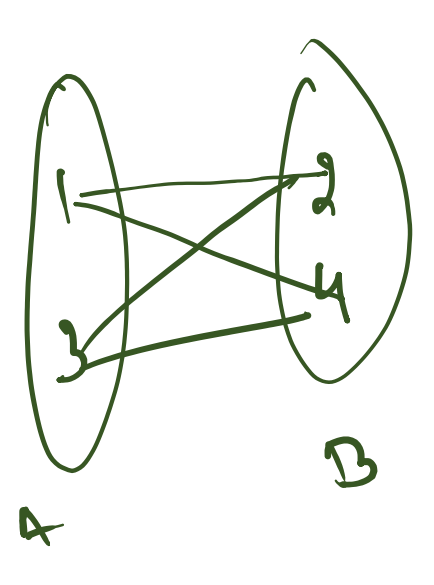
```
for (int i = 0; i < n; i++) {
            map.put(i, new ArrayList<>());
        }
        for (int i = 0; i < edges.length; i++) {
            int v1 = edges[i][0];
            int v2 = edges[i][1];
            map.get(v1).add(v2);
            map.get(v2).add(v1);
        }
```

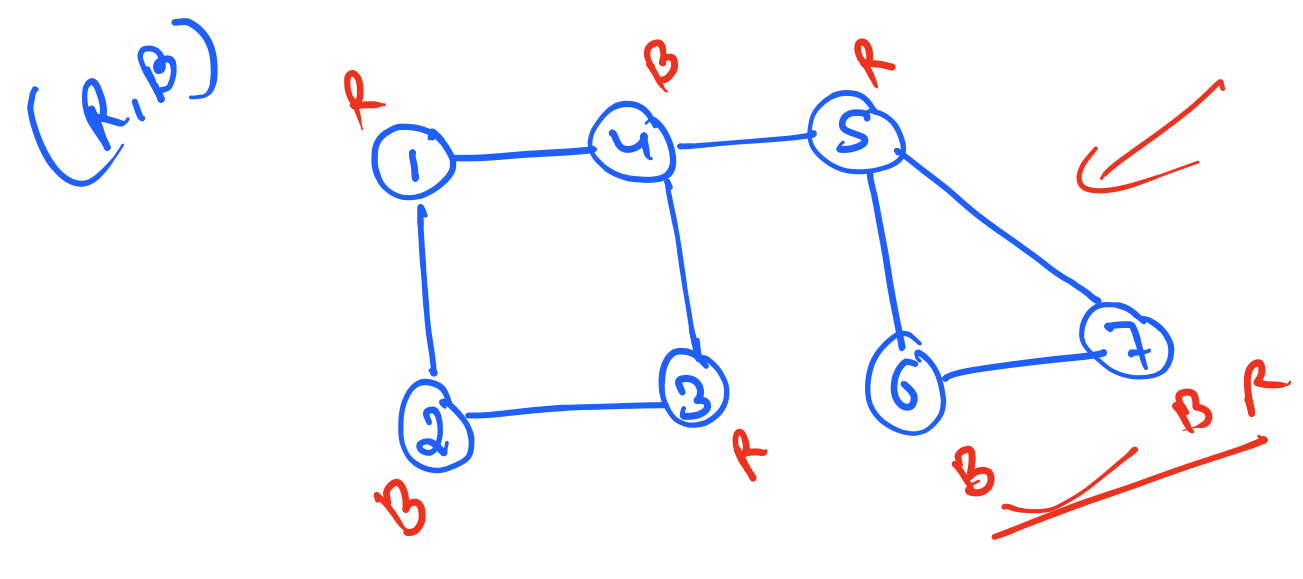

0
1
2
3
4
1
0 2 1
1 3
2 1

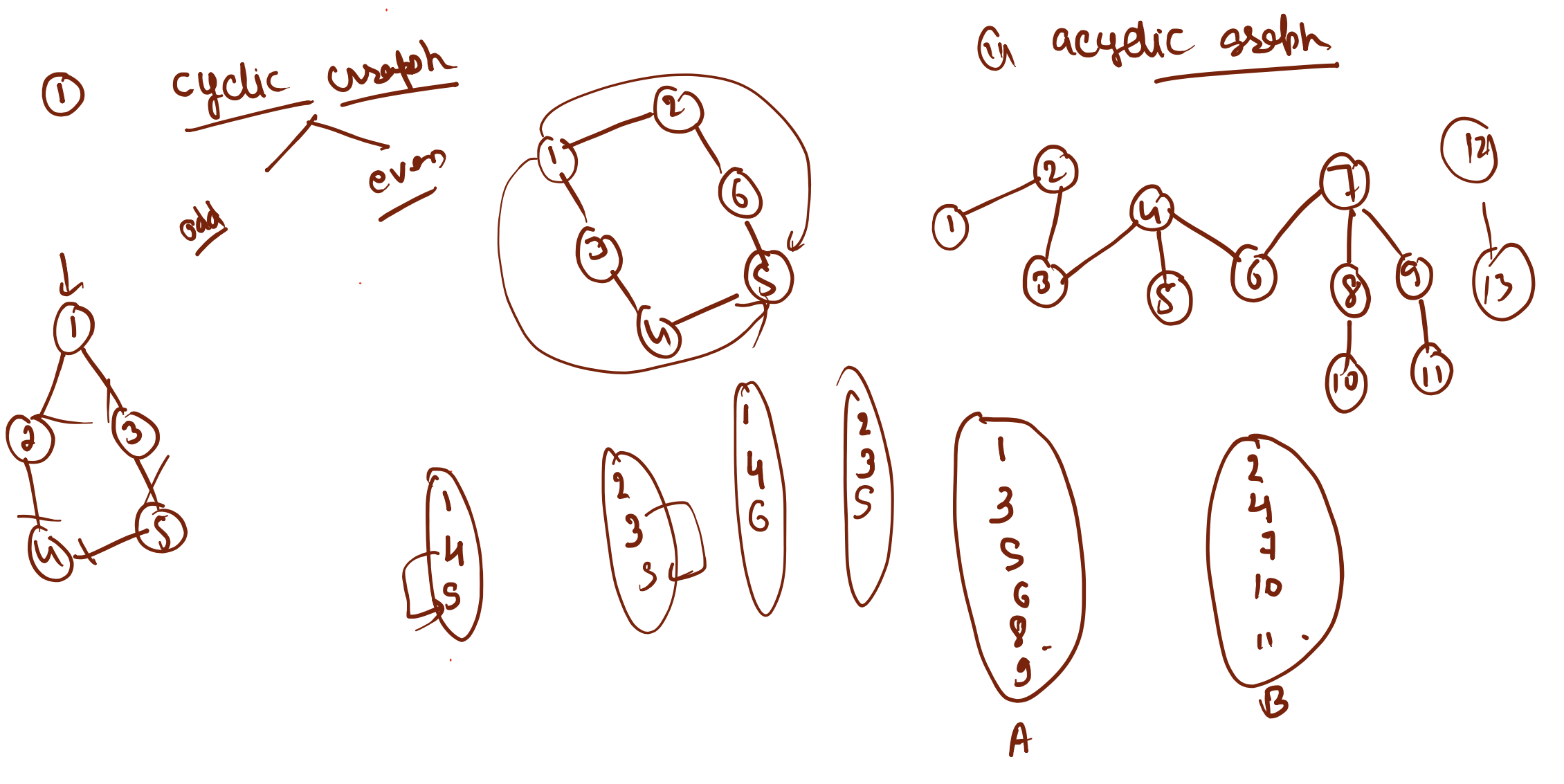
0 1
[[0,1],[1,2],[2,3],[1,3],[1,4]]

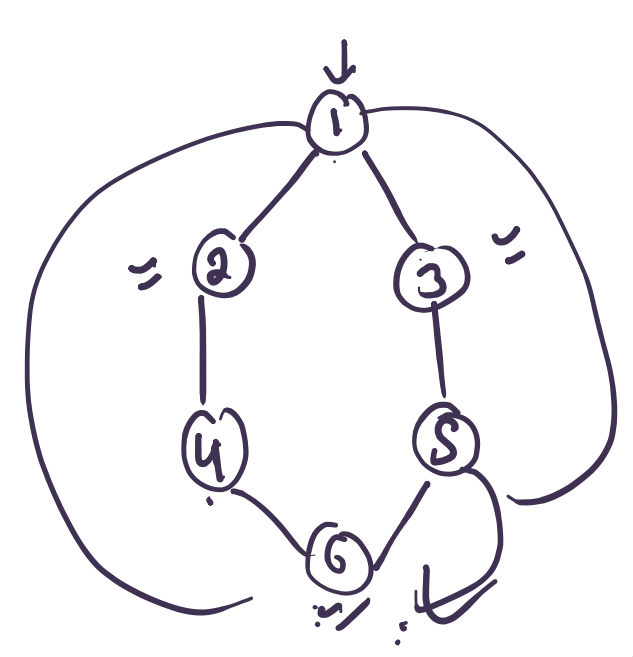






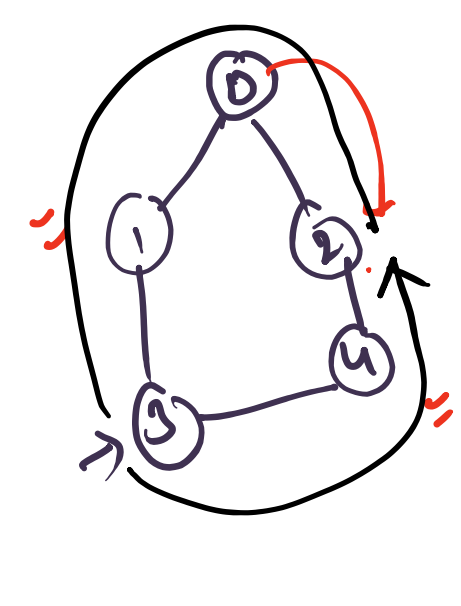

(R,B)
R
B
R
1
4
5
2
3
6
7
B
R
B
B R


①
cyclic graph
odd
even
1
2
3
4
5
1
2
6
3
4
5
1
4
5
2
3
3
1
4
6
2
3
5
1
3
5
6
8
9
A
G acyclic graph
1
2
3
4
5
6
7
8
9
10
11
12
13
2
4
7
10
11
B



~~1,0~~ | ~~2,1~~ | ~~3,1~~ | ~~4,2~~ | ~~5,2~~ | ~~6,3~~ | ~~6,2~~

1	0
2	1
3	1
4	2
5	2
6	3



~~3,0~~ | ~~1,1~~ | ~~4,1~~ | ~~0,2~~ | ~~2,2~~ | 2,3

3	0
1	1
4	1
0	2
2	2

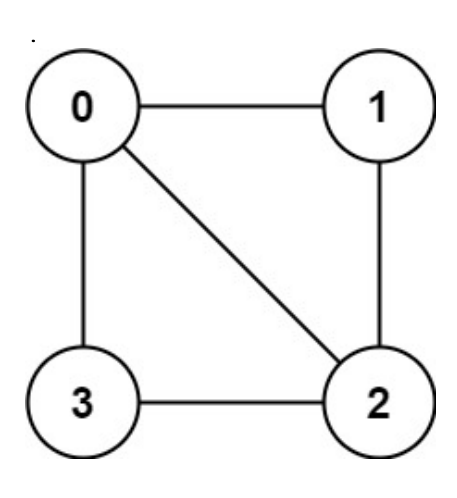
```
public boolean isBipartite(int[][] graph) {
        Queue<BipartitePair> q = new LinkedList<>();
        HashMap<Integer, Integer> visited = new HashMap<>();
        for (int i = 0; i < graph.length; i++) {// vtx
            if (visited.containsKey(i)) {
                continue;
            }
            q.add(new BipartitePair(i, 0));
            while (!q.isEmpty()) {
                // 1. remove
                BipartitePair rp =q.poll();
                // 2. Ignore if Allready visited
                if(visited.containsKey(rp.vtx)) {
                    if(visited.get(rp.vtx)!=rp.dis) {
                        return false;
                    }
                    else {
                        continue;
                    }
                }
                // 3. marked visited
                visited.put(rp.vtx, rp.dis);
                // 4. self work
                // 5. add unvisited nbrs
                for(int nbrs:graph[rp.vtx]) {
                    if(!visited.containsKey(nbrs)) {
                        q.add(new BipartitePair(nbrs, rp.dis+1));
                    }
                }
            }
        }
        return true;

    }
```

[0 1 2 3]

[1 2 3]

1 2

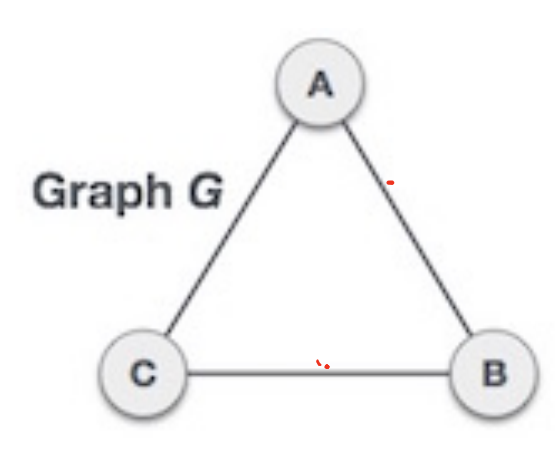
~~0,0~~ | ~~1,1~~ | ~~2,1~~ | ~~3,1~~ | ~~2,2~~ | 3,2

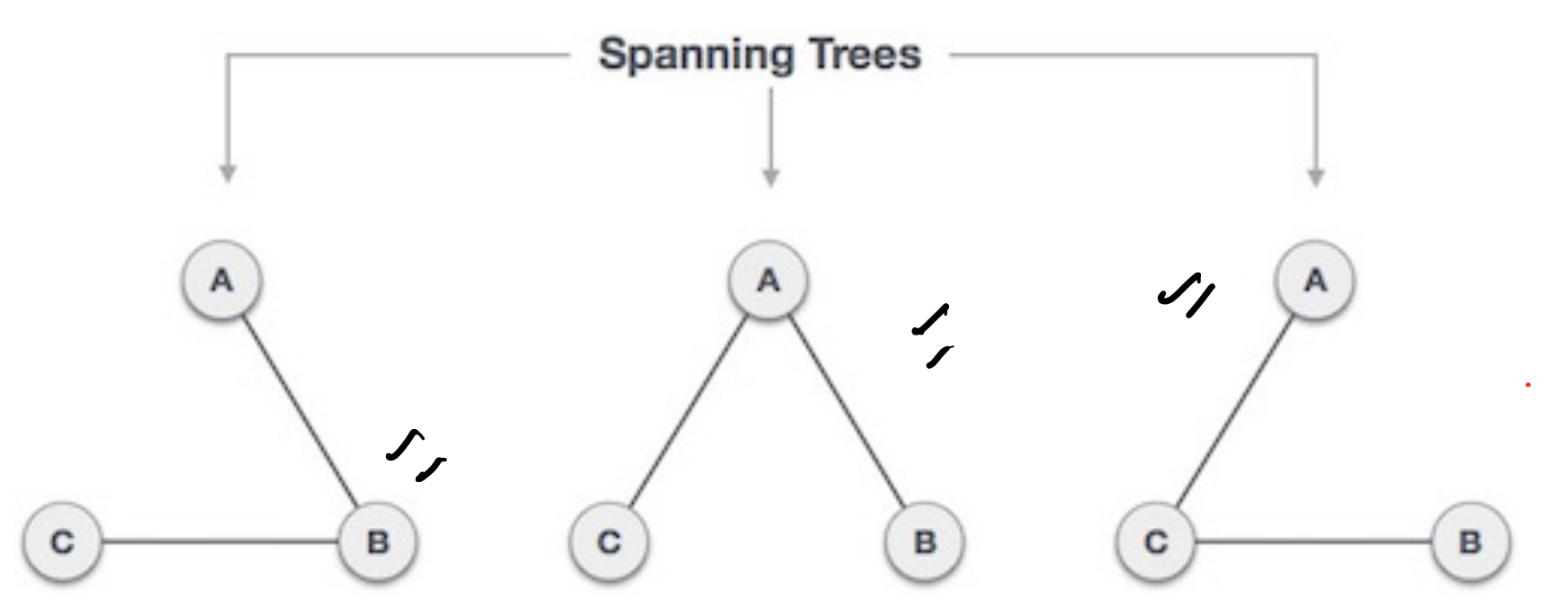


0	0
1	1
2	1
3	1

Spanning Tree

A spanning tree is a subset of Graph G, which has all the vertices covered with minimum ~~possible~~ number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..



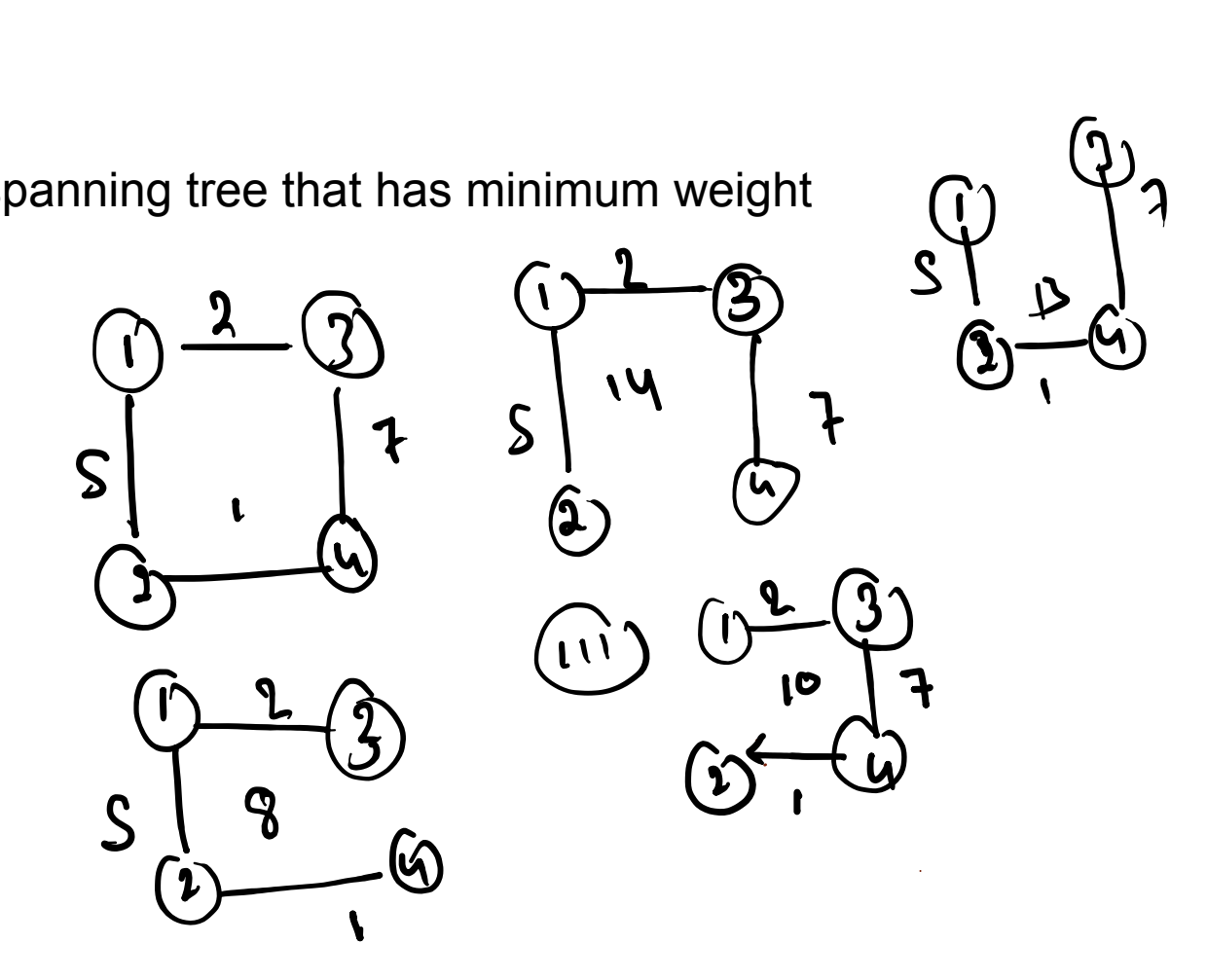

Spanning Trees
A
C
B
A
C
B
A
C
B

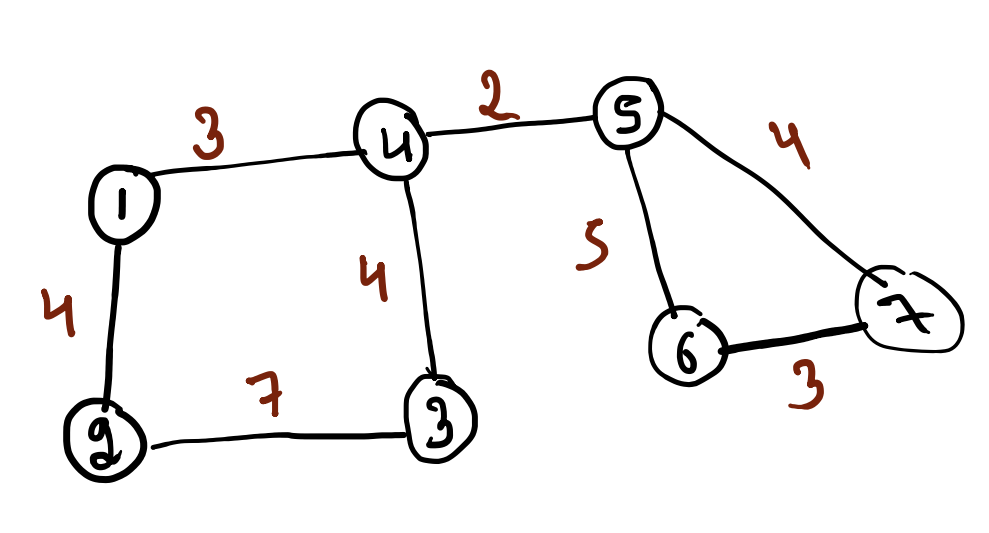
Properties of Spanning Tree

- A connected graph G can have more than one spanning tree.
- Spanning Tree has n-1 edges, where n is the number of Node (Vertices)
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops). ✗
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is minimally connected.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is maximally acyclic.

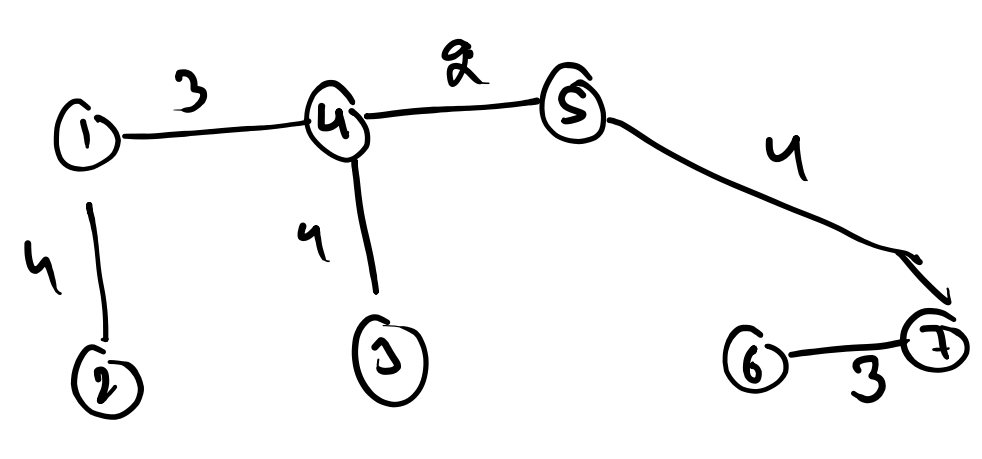
Minimum Spanning Tree (MST)
In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.
Kruskal's Algorithm

Prim's Algorithm
Both are greedy algorithms.

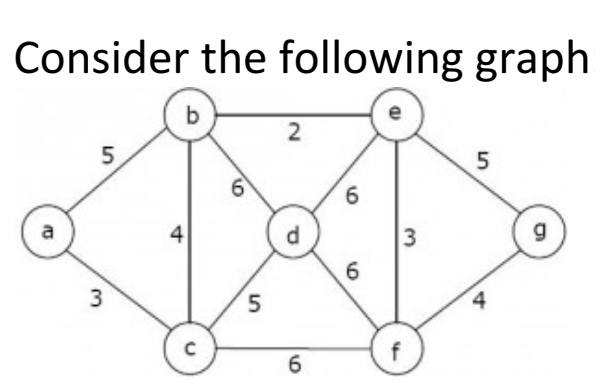

1
2
3
5
1
7
4
3
1
2
3
5
14
7
2
4
1
5
13
3
1
4
7
(iii)
1
2
3
10
7
2
1
4
1
2
3
5
8
2
1
4


3
2
4
1
4
5
4
4
7
5
6
7
3
g
3

2+3+3+4+4+4 = 20


1
3
4
2
5
4
4
4
2
3
6
3
7

Consider the following graph:


a
b
c
d
e
f
g
5
4
3
6
2
4
6
5
6
3
5
6
4

Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?
(A) (b,e)(e,f)(a,c)(b,c)(f,g)(c,d)
(B) (b,e)(e,f)(a,c)(f,g)(b,c)(c,d)
(C) (b,e)(a,c)(e,f)(b,c)(f,g)(c,d)
(D) (b,e)(e,f)(b,c)(a,c)(f,g)(c,d)

~~D.S.U~~ ✗

Prims

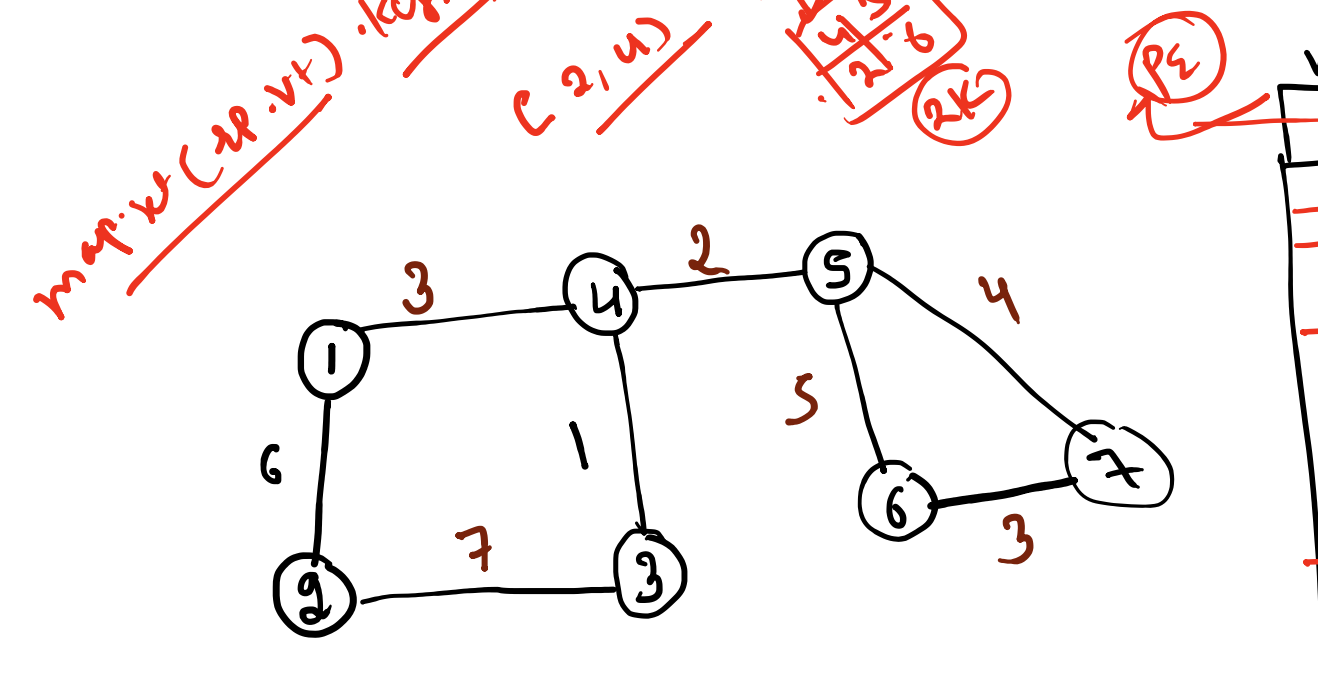
B.F.S

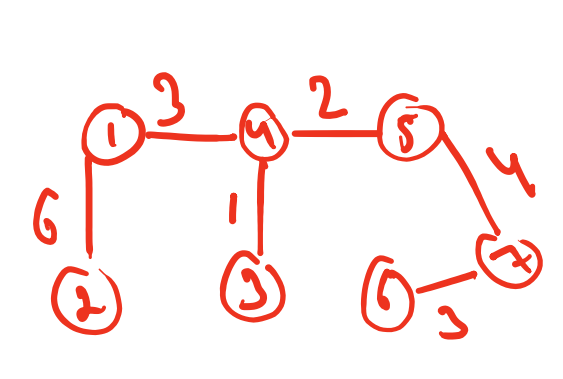
map.get(rp.vtx).keySet

(2,4)

map.get(rp.vtx).get(nbr)

PQ


1
3
4
2
5
4
6
1
5
7
6
7
3
g
3


1
3
4
2
5
4
6
1
2
3
6
3
7

1 1 @ 0
4 1 @ 3
3 4 @ 1
5 4 @ 2
7 5 @ 4
6 7 @ 3
2 1 @ 6

vtx	acq vtx	cost
~~1~~	~~1~~	~~0~~
~~2~~	~~1~~	~~6~~
~~4~~	~~1~~	~~3~~
~~3~~	~~4~~	~~1~~
~~3~~	~~4~~	~~2~~
~~2~~	~~3~~	~~7~~
~~6~~	~~5~~	~~5~~
~~7~~	~~5~~	~~4~~
~~6~~	~~7~~	~~3~~

1. Remove
2. ignore
3. visited
4. self work
5. Add nbrs

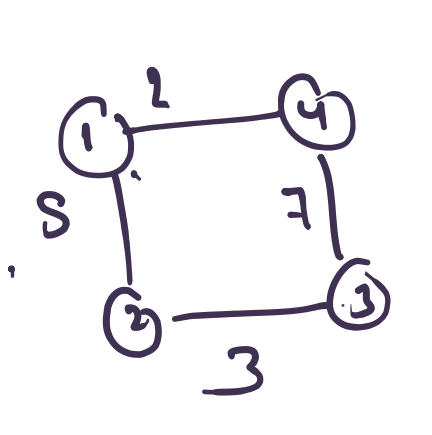
1 4 3
5 7 6
2

visited

```
public void Prims() {
        PriorityQueue<PrimsPair> pq = new PriorityQueue<>();
        HashSet<Integer> visited = new HashSet<>();
        pq.add(new PrimsPair(1, 1, 0));
        while (!pq.isEmpty()) {
            // 1. remove
            PrimsPair rp =pq.poll();
            // 2. Ignore if Already visited
            if(visited.contains(rp.vtx)) {
                continue;
            }
            // 3. marked visited
            visited.add(rp.vtx);
            // 4. self work
            System.out.println(rp);
            // 5. add nbrs
            for(int nbrs:map.get(rp.vtx).keySet()) {
                if(!visited.contains(nbrs)) {
                    int cost=map.get(rp.vtx).get(nbrs);
                    pq.add(new PrimsPair(nbrs, rp.vtx, cost));
                }
            }
        }
    }
```

1 1 @ 0
4 1 @ 2
2 1 @ 5
3 2 @ 3

{2,4}

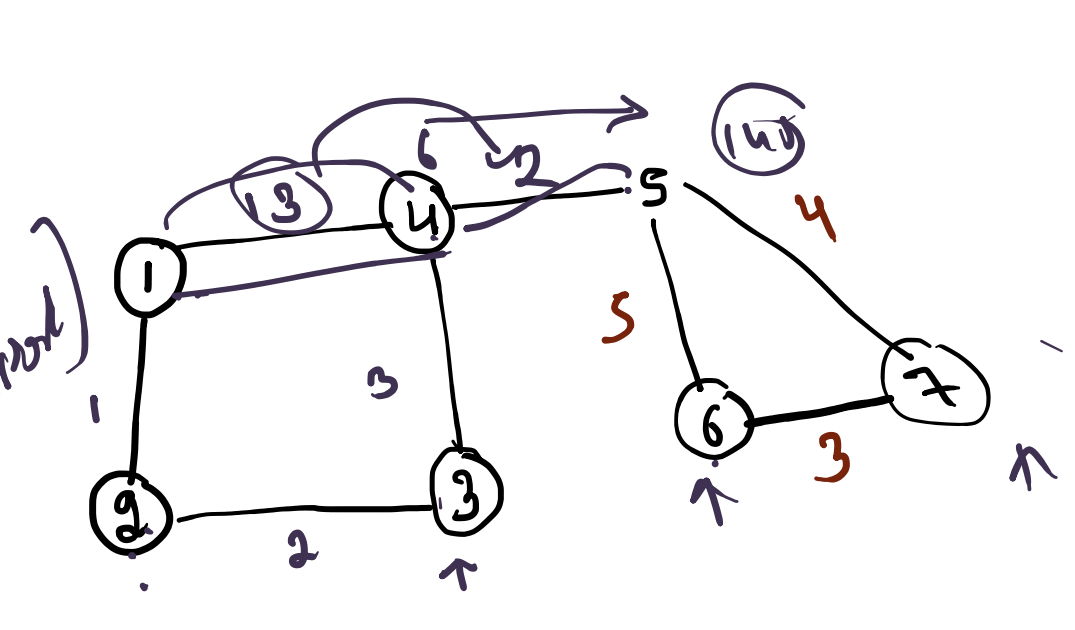


1 4 2 3

visited

vtx	acq vtx	cost
1	1	0
~~2~~	~~1~~	~~5~~
~~4~~	~~1~~	~~2~~
3	4	7
~~3~~	~~2~~	~~3~~

SSSP
(i) Dijkstra
(ii) Bellman ford


13
6
2
14
1
4
5
4
1
3
5
6
7
3
g
2
3

1 1 @ 0
2 12 @ 1
3 123 @ 3
4 1234 @ 6
5 12345 @ 8
7 123457 @ 12
6 123456 @ 13

vtx	1→4→min cost path	cost
1	1	0
2	12	1
4	14	~~13~~
3	123	3
4	1234	6
5	12345	8
6	123456	13
7	12357	12
6	123576	15

1 2 3 4
5 7 6